

Partnr Network - Talent Layer for Borderless Societies

 app.partnr.network/global-placement-program/tasks/017808284a724a29966d

Partnr Network

Objective

Build a fully responsive, animated personal portfolio website from scratch. You will learn to use modern frontend frameworks like React or Next.js, implement advanced CSS effects like parallax scrolling and on-scroll animations, and deploy a live website to a public URL. This project is essential for showcasing your skills to potential employers in a professional, polished format.

Description

Overview

Design, build, and deploy a **personal portfolio website** from scratch. This is not a template-based project — every line of code must be written (or AI-assisted and understood) by you. The final product must be a **live, publicly accessible website** that showcases who you are as a developer.

Your portfolio should reflect your personality. Close your eyes and imagine what feels like *you* — fonts, colors, spacing, motion style, visual tone, and voice. Bring originality instead of copying a generic style.

At the same time, prioritize shipping your first complete iteration quickly. Get a strong version live first, then spend extra time polishing details, motion quality, and visual refinement.

This assignment is divided into **5 major phases**, each broken down into **sub-phases** with step-by-step instructions. Read every sub-phase carefully before starting. Do not skip ahead — each phase builds on the previous one.

Your portfolio must include:

- A fully responsive layout (mobile + tablet + desktop)
- At least **1 parallax scrolling effect**
- At least **3 on-scroll animations**
- Live deployment on a free hosting platform
- Clean, readable, well-structured code pushed to a public GitHub repository

PHASE 1 — Environment Setup & Project Planning

Goal: Have your tools installed, your stack decided, and your content planned before writing a single line of production code.

Phase 1.1 — Choose Your AI IDE

You will use an AI IDE throughout this project. The goal is not just generating code faster, but learning how to direct agents, use skills/rules, and validate output.

Pick one primary AI IDE and stay on it for this assignment:

Option A — Cursor (*recommended starting point*)

- **Download:** <https://cursor.com>
- Great for chat + inline edits, fast iteration, and workflow customization via rules/skills/hooks.

Option B — Antigravity

- Supports AI-assisted coding workflows and agent-style development.
- Use it if you already prefer its workflow and can move quickly with it.

Option C — Codex

- Useful for prompt-first coding, refactors, and task execution in a codebase.
- Best if you are comfortable giving structured instructions and reviewing diffs.

Option D — Claude Code

- Strong for reasoning-heavy coding tasks and codebase-level edits.
- Works well when you provide clear specs and verification steps.

Option E — Windsurf (by Codeium)

Agentic IDE experience with autonomous multi-file edits.

Option F — GitHub Copilot (in VS Code)

Useful for inline completions and short assistant interactions.

| Install one primary IDE. Switching tools repeatedly slows you down.

How to use your AI IDE for this project (important)

- Ask it to generate a plan for each phase, then execute task-by-task (not one giant prompt).
- Ask it to **find and use project SKILL.md files** when available.
- Ask it to check and follow **Cursor rules** (for Cursor users: `.cursor/rules/` and `AGENTS.md` if present).
- If you are using **Codex**, explicitly prompt it to discover guidance files first, then implement in small verified diffs.
- Use it to scaffold components, but you must review and understand every change.
- After each feature, ask for a quick self-review: accessibility, responsiveness, and performance risks.

Phase 1.2 — Choose Your Technology Stack

Before writing code, choose one stack and stick to it for the full assignment.

Stack A — React + Vite (*Recommended for most students*)

Use this if: You want fast setup, fast iteration, and a component-based workflow.

Why this is recommended:

- Quick local development cycle
- Works smoothly with most animation/motion libraries
- Easy to combine with Tailwind, shadcn/ui, Magic UI, and Aceternity UI

Stack B — Next.js (*For advanced students*)

Use this if: You are comfortable with React and want stronger SEO and production-style architecture.

Why choose this:

- Better SEO defaults and routing model
- Great fit for content-heavy portfolios
- Works well with the same UI and animation ecosystem

| Recommendation: use **React + Vite** unless you already know Next.js well.

AI-first build acceleration (required mindset)

To ship a polished portfolio quickly, prioritize **ready-made UI components** and adapt them to your brand:

- **AcertinityUI / Aceternity UI:** <https://ui.aceternity.com>
- **shadcn/ui:** <https://ui.shadcn.com>
- **Magic UI:** <https://magicui.design>

Use your AI IDE to:

- discover suitable components for each section (hero, projects, testimonials, contact),
- adapt motion, colors, spacing, and typography,
- keep visual consistency across the whole site.

Phase 1.3 — Plan Your Content & Create a Wireframe

Do this before writing any HTML or JSX. Planning your layout upfront prevents major rewrites later.

Step 1 — Decide your portfolio sections

Your portfolio **must contain all 5 required sections:**

Section	What It Must Include
---------	----------------------

Hero	Your name, your title/role (e.g. "Full Stack Developer"), a one-line description of yourself, a primary CTA button (e.g. "View My Work" or "Download Resume"), and optionally your photo or an illustration
-------------	---

Section	What It Must Include
About	A short bio (3–5 sentences), your photo or avatar, a brief mention of your background, what you're currently learning or working on
Skills	A visual display of technologies you know (e.g. HTML, CSS, JavaScript, React, Python, etc.) using icons, badges, or progress bars
Projects	At least 3 real projects with: project title, 2–3 line description, tech stack used, a link to the GitHub repo, and optionally a live demo link
Contact	Your email as a clickable <code>mailto:</code> link, GitHub profile link, LinkedIn profile link, and optionally a contact form

Step 2 — Create a wireframe

A wireframe is a simple black-and-white sketch of your page layout. It does not need to be polished — boxes and labels are enough.

Wireframe must show:

- The order of sections from top to bottom
- Where the navigation bar sits and what links it contains
- Rough layout of each section (columns, card grids, text positions)
- Where animated elements will appear (note these with a star or label)

Do not proceed to Phase 2 until your wireframe is complete and you have a clear picture of every section's layout.

PHASE 2 — Core Development: Structure & Styling

Goal: Build the complete HTML/JSX structure of your portfolio and apply base styling so every section is visible and responsive — without any animations yet.

Phase 2.1 — Initialize Git and Push to GitHub

Set up version control before building major features. Your project must be in a Git repository and pushed to a public GitHub repository. You should commit after each meaningful milestone.

Phase 2.2 — Set Up Your CSS / Styling System

Choose one primary styling approach for the whole project. Keep it consistent.

Option A — Tailwind CSS (*Recommended*)

- **URL:** <https://tailwindcss.com>
- **Why:** Fastest path for building polished UI with consistent spacing and responsive behavior.

Option B — SCSS / Sass

- **URL:** <https://sass-lang.com>
- **Why:** Good if you prefer handcrafted styling with variables and modular structure.

Option C — CSS Modules

- **URL:** Built into Vite and Next.js — no install needed.
- **Why:** Component-scoped CSS with low risk of style collisions.

Typography Setup (Required for all styling options)

Use **Google Fonts** (<https://fonts.google.com>). Pick a clear display font + body font pair and use them consistently across all sections. Do not use default system fonts.

Phase 2.3 — Build the HTML/JSX Structure

Build all required sections from your wireframe and keep the structure component-first.

Required outcomes:

- Sticky/responsive navigation
- Hero, About, Skills, Projects, Contact, Footer
- Minimum 3 projects with clear descriptions + links
- Skills presented visually (not plain text)

Phase 2.4 — Make It Fully Responsive

Every section must work on mobile, tablet, and desktop. Use browser developer tools to test at various screen widths (e.g., 375px, 768px, 1280px). Ensure no content overflows, images scale correctly, and no horizontal scrollbars appear on mobile.

PHASE 3 — Advanced Animations & Motion Effects

Goal: Layer animations onto your already-built structure. Every animation should feel purposeful, not decorative. Performance matters — a janky animation is worse than no animation.

Critical performance rule: Always animate using `transform` and `opacity` only. Never animate `top`, `left`, `width`, `height`, `margin`, or `padding`.

Phase 3.1 — Install and Configure Your Motion Library

Choose at least one primary motion library. Recommended options include **Framer Motion**, **GSAP + ScrollTrigger**, or **AOS**.

Phase 3.2 — Implement Parallax Scrolling

You must include at least one parallax effect. Use a library like **react-scroll-parallax** or implement it with **GSAP ScrollTrigger**. The effect should be subtle and applied to decorative background elements.

Phase 3.3 — On-Scroll Animation Requirements

You must include at least **3 on-scroll animations**. Examples include fade/slide reveals for sections, staggered card reveals for projects, or skill bar fill animations.

Phase 3.4 — Accessibility Requirement for Animations

Add the `prefers-reduced-motion` media query to your global CSS to disable animations for users with this accessibility setting enabled. This is **required**.

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: 0.01ms !important;  
    animation-iteration-count: 1 !important;  
    transition-duration: 0.01ms !important;  
    scroll-behavior: auto !important;  
  }  
}
```

PHASE 4 — Assets, Polish & Performance

Goal: Source all visual assets legally, refine the visual design, and optimize the site so it loads fast and scores well on Lighthouse.

Phase 4.1 — Source Free & Legal Visual Assets

Every image, icon, and illustration must be either self-made or from a copyright-free platform like **Unsplash**, **Undraw**, or **Iconify**. **Do not pull images from Google Search.**

Phase 4.2 — Image Optimization

All images must be optimized before deployment. Convert images to **WebP** format using a tool like **Squoosh**. Ensure hero images are under 300KB and other images are under 150KB. Use `loading="lazy"` on all images below the fold.

Phase 4.3 — Run a Lighthouse Audit

Use the built-in Chrome Lighthouse tool to audit your deployed site. You must meet the minimum required scores.

Category	Minimum Score
Performance	80
Accessibility	90
Best Practices	90
SEO	85

Ensure your HTML `<head>` includes a proper `<title>`, `<meta name="description">`, and `<meta name="viewport">`.

PHASE 5 — Deployment & Submission

Goal: Get your portfolio live on the internet with a public URL, and submit all required materials.

Phase 5.1 — Choose a Deployment Platform

Deploy your site using a free platform. **Vercel** is recommended for React/Next.js projects. **Netlify** is a great alternative.

Phase 5.2 — Deploy Your Site

Connect your GitHub repository to your chosen deployment platform (Vercel or Netlify). The platform will auto-detect your framework and build settings. Once configured, every push to your `main` branch will trigger a new deployment automatically.

Phase 5.3 — Post-Deployment Verification Checklist

After deployment, test your **live URL** thoroughly. Check that all links work, images load, animations are smooth, and there are no console errors. Verify functionality on both desktop and a real mobile device.

Core Requirements

1.

The entire website must be fully responsive and visually polished on three main breakpoints: mobile (375px), tablet (768px), and desktop (1280px).

Verification:

- ✓ No horizontal scrollbars appear at any of the specified widths.
- ✓ Text is readable and does not overflow its containers.
- ✓ Navigation adapts correctly (e.g., collapses into a hamburger menu on mobile).
- ✓ Images and other media scale properly without distortion.

2.

The portfolio must include all five required content sections, populated with your personal information.

Verification:

- ✓ **Hero:** Contains your name, title, a short bio, and a call-to-action button.
- ✓ **About:** Contains a 3-5 sentence bio and a photo/avatar.
- ✓ **Skills:** Visually displays your technical skills (e.g., with icons or badges).
- ✓ **Projects:** Lists at least 3 projects with descriptions, tech stacks, and links.
- ✓ **Contact:** Includes clickable links for email, GitHub, and LinkedIn.

3.

The 'Projects' section must feature at least three distinct projects.

Verification:

- ✓ At least 3 project entries are present.
- ✓ Each project must have a title, a 2-3 line description, and a list of technologies used.
- ✓ Each project must include a functional link to its public GitHub repository.

4.

The website must implement at least one parallax scrolling effect.

Verification:

- ✓ At least one element on the page moves at a different speed than the main scroll speed, creating a depth effect.
- ✓ The effect is smooth and does not cause performance issues (jank).
- ✓ The effect is preferably applied to a background or decorative element.

5.

The website must implement at least three distinct on-scroll animations.

Verification:

- ✓ At least three elements or sections animate into view as the user scrolls down the page.
- ✓ Animations can include fade-ins, slide-ins, staggered reveals, or other similar effects.
- ✓ Animations trigger only when the element enters the viewport.

6.

The completed portfolio website must be deployed and publicly accessible via a live URL.

Verification:

- ✓ The provided URL is live and resolves to the portfolio website.
- ✓ The site is served over HTTPS.

7.

The deployed website must meet minimum Google Lighthouse scores when audited on a desktop device.

Verification:

Category	Minimum Score
Performance	80
Accessibility	90
Best Practices	90
SEO	85

8.

The application's CSS must respect the `prefers-reduced-motion` accessibility feature.

Verification:

- ✓ The global stylesheet must contain a `@media (prefers-reduced-motion: reduce)` block.
- ✓ Inside this block, CSS rules must effectively disable or significantly reduce animations and transitions for users who have this preference enabled in their operating system.

9.

The GitHub repository must contain a `README.md` file with specific information.

Verification:

- ✓ The `README.md` file exists at the root of the repository.
- ✓ It contains the author's name, project description, and tech stack.
- ✓ It includes a working link to the live deployed site.
- ✓ It provides clear, correct instructions for setting up and running the project locally.